

chap 05. C Operators

Contents

01 블랙박스 테스트 개요

02 기능 기반 테스트 케이스 생성 기법

03 시나리오 기반 테스트

04 테스트 단계

[프로젝트 XIII] 프로젝트 개발 코드에 대하여 블랙박스 테스트 실행하기

01 블랙박스 테스트 개요

Operators

- C divides the operators into the following groups:
 - Arithmetic operators
 - Assignment operators
 - Comparison operators
 - Logical operators
 - Bitwise operators (advanced)

Arithmetic Operators

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
++	Increment	$++x$
--	Decrement	$--x$

```
1 int x = 10;
2 int y = 3;
3
4 printf("%d\n", x + y); // 13
5 printf("%d\n", x - y); // 7
6 printf("%d\n", x * y); // 30
7 printf("%d\n", x / y); // 3
8 printf("%d\n", x % y); // 1
9
10 int z = 5;
11 ++z;
12 printf("%d\n", z); // 6
13 --z;
14 printf("%d\n", z); // 5
15
16 int a = 10;
17 int b = 3;
18 printf("%d\n", a / b); // Integer division, result is 3
19
20 double c = 10.0;
21 double d = 3.0;
22 printf("%f\n", c / d); // Decimal division, result is 3.333...
```

Arithmetic Operators

■ Incrementing and Decrementing

- Incrementing and decrementing are very common in programming, especially when working with counters, loops, and arrays

```
1 int x = 5;
2
3 ++x; // Increment x by 1
4 printf("%d\n", x); // 6
5
6 int x = 5;
7
8 --x; // Decrement x by 1
9 printf("%d\n", x); // 4
```

```
1 int peopleInRoom = 0;
2
3 // 3 people enter
4 peopleInRoom++;
5 peopleInRoom++;
6 peopleInRoom++;
7
8 printf("%d\n", peopleInRoom); // 3
9
10 // 1 person leaves
11 peopleInRoom--;
12
13 printf("%d\n", peopleInRoom); // 2
```

Assignment

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Comparison Operators

■ Comparison Operators

- Comparison operators are used to compare two values (or variables)
- The return value of a comparison is either 1 or 0, which means true (1) or false (0)

perator	Name	Example	Description
==	Equal to	<code>x == y</code>	Returns 1 if the values are equal
!=	Not equal	<code>x != y</code>	Returns 1 if the values are not equal
>	Greater than	<code>x > y</code>	Returns 1 if the first value is greater than the second value
<	Less than	<code>x < y</code>	Returns 1 if the first value is less than the second value
>=	Greater than or equal to	<code>x >= y</code>	Returns 1 if the first value is greater than, or equal to, the second value
<=	Less than or equal to	<code>x <= y</code>	Returns 1 if the first value is less than, or equal to, the second value

Comparison Operators

■ Examples

```
1 int age = 18;
2
3 printf("%d\n", age >= 18); // 1 (true), old enough to vote
4 printf("%d\n", age < 18);  // 0 (false)
5
6 int passwordLength = 5;
7
8 printf("%d\n", passwordLength >= 8); // 0 (false), too short
9 printf("%d\n", passwordLength < 8);  // 1 (true), needs more characters
```

Logical Operators

■ Logical Operators

- Logical operators are used to determine the logic between variables or values, by combining multiple conditions:

Operator	Name	Example	Description
&&	AND	<code>x < 5 && x < 10</code>	Returns 1 if both statements are true
	OR	<code>x < 5 x < 4</code>	Returns 1 if one of the statements is true
!	NOT	<code>!(x < 5 && x < 10)</code>	Reverse the result, returns 0 if the result is 1

```
1 int isLoggedIn = 1;
2 int isAdmin = 0;
3
4 printf("Regular user: %d\n", isLoggedIn && !isAdmin);
5 printf("Has access: %d\n", isLoggedIn || isAdmin);
6 printf("Not logged in: %d\n", !isLoggedIn);
```

Operator Precedence

■ Operator Precedence

- When a calculation contains more than one operator, C follows **order of operations** rules to decide which part to calculate first.

■ Order of Operations

- () - Parentheses
- *, /, % - Multiplication, Division, Modulus
- +, - - Addition, Subtraction
- >, <, >=, <= - Comparison
- ==, != - Equality
- && - Logical AND
- || - Logical OR
- = - Assignment

Challenge: Calculate the Total Cost of an Item

■ Instructions

- Inside main(), complete the following steps:
 1. Declare two int variables named itemPrice(50) and shippingCost(15), and assign them values
 2. Create an int variable named sum
 3. Calculate the total cost by adding itemPrice and shippingCost (store the result in sum)
 4. Print the total cost using printf

```
1 #include <stdio.h>
2
3 int main() {
4     // Write itemPrice here
5     // Write shippingCost here
6
7     // Write sum here (itemPrice + shippingCost)
8
9     // Print sum here
10
11     return 0;
12 }
```

```
1 #include <stdio.h>
2
3 int main() {
4     // Write itemPrice here
5     // Write shippingCost here
6
7     // Write sum here (itemPrice + shippingCost)
8
9     // Print sum here
10
11     return 0;
12 }
```

User Input

■ user Input

- To get user input, you can use the scanf() function:

```
1 // Create an integer variable that will
2 // store the number we get from the user
3 int myNum;
4
5 // Ask the user to type a number
6 printf("Type a number: \n");
7
8 // Get and save the number the user types
9 scanf("%d", &myNum);
10
11 // Output the number the user typed
12 printf("Your number is: %d", myNum);
```

```
1 // Create an int and a char variable
2 int myNum;
3 char myChar;
4
5 // Ask the user to type a number AND a character
6 printf("Type a number AND a character and press enter: \n");
7
8 // Get and save the number AND character the user types
9 scanf("%d %c", &myNum, &myChar);
10
11 // Print the number
12 printf("Your number is: %d\n", myNum);
13
14 // Print the character
15 printf("Your character is: %c\n", myChar);
```

Code challenge

■ Exchange rate calculation

■ Instruction

■ Inside main(), complete the following steps:

1. Declare two int variables named nDollar , nWon
2. print input current Won per dollar
3. get from user nDollar
4. print input total Won
5. get from user nWon
6. Calculate and print exchanged dollar

```
1 #include <stdio.h>
2 int main(){
3     int
4     int
5
6     pr
7     sca
8     pr
9     sca
10
11     printf("won to dalar : %f\n", (float)nWon / nDalar );
12 }
```

Thank You !